Universidad Central

Arquitectura de Sistemas I

Informe de las semanas 13 y 14

Persistencia, servicios y API del corte vertical

Proyecto Arquitectura de Sistemas

Adopción responsable de mascotas

Proyecto:	Sistema de Apoyo a la Decisión para la Adopción Responsable de Mascotas mediante Inteligencia Artificial
Docente:	Elias Buitrago Bolivar
Integrantes:	Juan Diego Rojas y Adrián Camaño
Fecha:	10 de noviembre de 2026
Estado:	12/12 pruebas de persistencia y 39/39 pruebas de API

1. Resumen de la entrega

Este documento reúne el trabajo de las semanas 13 y 14 sobre el mismo corte vertical. La semana 13 construyó la base de datos y la capa de persistencia; la semana 14 agregó los servicios de aplicación, la API REST, los contratos y la verificación de los requisitos no funcionales. Las dos semanas conservan los mismos dos requisitos funcionales priorizados.

Semana	Trabajo realizado	Resultado	Evidencia
13	Esquema, reglas de integridad, repositorio y datos de prueba	12/12 pruebas de persistencia	`evidencias/salida_pruebas.txt`
14	Servicios, API FastAPI, contratos, pruebas funcionales y RNF	39/39 pruebas; 100 % de cobertura en servicios	`salida_pruebas_semana14.txt`, `cobertura.txt`, `rnf_desempeno.txt` y `demo_api.txt`

Orden	Requisito funcional	Resultado observable
1	**RF-05 — Gestión de la ficha de mascota**	El refugio crea, edita y da de baja una ficha; recibe su identificador y su estado (*incompleta* o *disponible*).
2	**RF-34 — Consulta del catálogo**	El adoptante lista las mascotas disponibles con filtros por especie, tamaño y edad, con paginación y orden estable.

Stack: Python 3.12.10, FastAPI 0.162.2, Pydantic v2, Uvicorn 0.54.0 y SQLite 3.49.1. SQLite se usa para reproducir el corte sin instalar un servidor; la base objetivo del proyecto sigue siendo PostgreSQL (ADR-03/ADR-10).

2. Contexto y alcance

2.1. El caso de estudio

El sistema nace en la asignatura *Práctica de Ingeniería*. Los refugios registran las fichas de los animales que tienen en adopción y los adoptantes consultan ese catálogo para encontrar una mascota compatible con su hogar. En **Arquitectura de Sistemas I** se retoma como caso real para aplicar decisiones de arquitectura; no se construye el sistema completo, sino un **corte vertical**.

Un corte vertical es una porción pequeña del sistema que se construye de punta a punta. En este proyecto avanza en tres momentos: la semana 13 cubre la base de datos y la persistencia; la semana 14, los servicios y la API; la semana 3 del prototipo, la interfaz, la autenticación y el cierre.

2.2. Los dos requisitos funcionales

- **RF-05.** El refugio crea, edita y da de baja la ficha de una mascota. La ficha incluye especie, raza aproximada, edad, tamaño, nivel de energía, historial de salud y comportamiento, experiencia requerida, aptitud para convivir con menores y otras mascotas, y de 1 a 5 fotografías.
- **RF-34.** El adoptante consulta el catálogo de mascotas disponibles y lo filtra por especie, tamaño y rango de edad, con paginación y orden por fecha de publicación.

Se eligieron porque **RF-05 produce la información y RF-34 la consume**. Juntos forman el recorrido mínimo del negocio —publicar una mascota y consultarla— y comparten las tablas *refugio*, *mascota* y *fotografia*.

2.3. Reglas de negocio del corte

Regla	Enunciado	Dónde se aplica
RN-22	Una ficha está completa cuando tiene los campos obligatorios y entre 1 y 5 fotos JPG/PNG; solo entonces pasa a *disponible*.	Repositorio, triggers y API
RN-03	Solo un refugio verificado puede publicar fichas.	Repositorio y API (ERR-009)
RN-15	Una ficha adoptada o dada de baja no aparece en el catálogo.	Filtro del catálogo y baja lógica

2.4. Fuera del alcance

Autenticación y roles, interfaz web, almacenamiento binario de fotos, motor de recomendaciones, solicitudes y decisiones de adopción, seguimiento posadopción, notificaciones, indicadores, chatbot y reconocimiento de imágenes. La migración definitiva a PostgreSQL se hará al cerrar el prototipo.

3. Semana 13 — Base de datos y persistencia

3.1. Modelo mínimo de datos

Entidad	Clave	Campos principales	Relación	RF
refugio	id (PK)	nombre, localidad, estado_verificacion (pendiente, verificado, rechazado)	1 : N con mascota	RF-05, RF-34
mascota	id (PK)	refugio_id (FK), nombre, raza, especie, edad_meses, tamano, nivel_energia, experiencia_requerida, apta_con_menores, apta_con_mascotas, historial_salud, historial_comportamiento, estado, fecha_publicacion	N : 1 con refugio; 1 : N con fotografia	RF-05, RF-34
fotografia	id (PK)	mascota_id (FK), archivo, orden (1 a 5)	N : 1 con mascota; UNIQUE (mascota_id, orden)	RF-05, RF-34

El esquema se recrea desde cero con `persistencia/schema.sql` y `persistencia/seed.sql`. La base de la semana 13 vive en `evidencias/prueba.db` y no se modifica desde la API.

3.2. Integridad y separación de capas

- **Llaves foráneas** de refugio a mascota y de mascota a fotografia, con borrado en cascada de las fotos.
- **CHECK** de dominios cerrados: especie, tamaño, energía, experiencia y estado; edad entre 0 y 300 meses.
- **UNIQUE** en (mascota_id, orden) y trigger que impide más de cinco fotos.
- **Triggers de publicación:** una ficha solo pasa a disponible si tiene al menos una foto y el refugio está verificado (RN-03 y RN-22).
- **Transacciones** con *BEGIN IMMEDIATE* y *ROLLBACK* ante error.
- **Repositorio único:** todo el SQL está en `persistencia/repositorio.py`; los servicios y la API no construyen consultas.
- **Catálogo:** solo fichas disponibles de refugios verificados, sin datos personales del adoptante.
- **Datos de fotos:** en este corte se guarda la ruta del archivo; el binario se incorporará más adelante.

3.3. Contrato del repositorio

Operación	Entrada	Resultado	Uso
`crear_ficha`	refugio_id, ficha de RN-22, 0–5 fotos	id real; estado incompleta o disponible	RF-05: persistir y devolver el identificador.
`obtener_por_id`	mascota_id	ficha con refugio y fotos, o None	RF-05: verificar el registro; representar la ausencia.
`actualizar_ficha`	mascota_id, refugio_id, cambios, fotos opcionales	ficha actualizada y estado recalculado	RF-05: editar y volver a publicar.
`dar_de_baja`	mascota_id, refugio_id	baja lógica (estado = baja)	RF-05: retirar la ficha del catálogo.
`listar_catalogo`	especie, tamano, edad_min, edad_max, pagina, por_pagina	total, pagina, porPagina, mascotas	RF-34: consulta paginada y filtrada.

3.4. Datos iniciales

Mascota	Refugio	Especie / tamaño	Estado	¿Catálogo?	Motivo
1 — Luna	1 — Patitas (verificado)	perro / grande	disponible	**Sí**	Ficha completa.
2 — Milo	1 — Patitas (verificado)	gato / pequeño	disponible	**Sí**	Ficha completa.
3 — Rocky	1 — Patitas (verificado)	perro / mediano	disponible	**Sí**	Ficha completa.
4 — Nube	2 — Colitas (pendiente)	gato / pequeño	disponible	*No*	Refugio no verificado.
5 — Toby	1 — Patitas (verificado)	perro / pequeño	incompleta	*No*	Sin fotografía.
6 — Canela	1 — Patitas (verificado)	perro / mediano	adoptada	*No*	Ya adoptada.

Catálogo esperado con los datos iniciales: **Luna (1), Rocky (3), Milo (2)**, ordenados por `fecha_publicacion` ascendente y, a igual fecha, por `id`.

3.5. Resultados de persistencia

Las 12 pruebas de la semana 13 se ejecutan con:

```
cd persistencia
$env:PYTHONUTF8 = "1"
$env:PYTHONIOENCODING = "utf-8"
python prueba_persistencia.py
```

#	Prueba	Resultado
1	Crear ficha en refugio verificado y leerla desde una conexión nueva	Pasa — id 7, disponible, 2 fotos.
2	Rechazar ficha de refugio no verificado	Pasa — ERR-009; 0 filas nuevas.
3	Rechazar datos inválidos	Pasa — ERR-004; 0 filas nuevas.
4	Consultar un identificador inexistente	Pasa — devuelve None.
5	Violar la FK de refugio	Pasa — FOREIGN KEY constraint failed.
6	Ficha sin fotos queda incompleta y fuera del catálogo	Pasa — estado incompleta.
7	Catálogo con orden, filtros y paginación	Pasa — total 4, orden [1, 3, 2, 7].
8	Actualizar, publicar y dar de baja	Pasa — incompleta → disponible → baja.
9	Rechazar más de cinco fotografías	Pasa — repositorio y trigger.
10	Triggers de publicación	Pasa — refugio pendiente y ficha sin foto.
11	Catálogo y ficha sin datos personales	Pasa — 0 campos personales.
12	20 consultas consecutivas al catálogo	Pasa — 0,53 ms promedio; 0,66 ms máximo.

Resumen: 12/12 pruebas superadas. La medición de tiempos de la semana 13 es una línea base con 8 fichas y no sustituye la prueba de carga de RNF-05. La evidencia completa está en `evidencias/salida_pruebas.txt`.

4. Semana 14 — Servicios y API

4.1. Arquitectura por capas

`HTTP` → `API` → `Servicio de aplicación` → `Repositorio` → `SQLite`

Capa	Archivo	Responsabilidad
API	`api/main.py`	Recibir HTTP, validar la forma, delegar y traducir errores a códigos HTTP. No contiene reglas ni SQL.
Contratos	`api/esquemas.py`	Modelos Pydantic v2 de entrada y salida.
Inyección	`api/dependencias.py`	Crear el repositorio y los servicios; la base se toma de `DB_PATH`.
Servicios	`servicios/`	Coordinar el caso de uso, normalizar la respuesta y decidir la ausencia de una ficha.
Repositorio	`persistencia/repositorio.py`	Todo el SQL, las validaciones de dominio y las transacciones.

Esta separación permite probar los servicios con un doble del repositorio, sin abrir una base real, y mantiene el SQL fuera de la API.

4.2. Servicios de aplicación

Servicio	Caso de uso	Métodos
`ServicioMascotas`	RF-05	`crear`, `consultar`, `actualizar`, `dar_de_baja`
`ServicioCatalogo`	RF-34	`listar` (filtros, paginación y normalización a snake_case)

`ServicioMascotas.consultar` traduce la ausencia de una ficha a `FichaNoEncontrada` (ERR-011). `ServicioCatalogo` delega los filtros en el repositorio y convierte `porPagina` a `por_pagina` en la respuesta JSON.

4.3. Endpoints

Base local: `http://127.0.0.1:8000/api/v1`. Documentación interactiva: `http://127.0.0.1:8000/docs`.

RF	Método y ruta	Servicio	Éxito	Errores posibles
RF-05	`POST /mascotas`	`ServicioMascotas.crear`	201 ficha creada	400 ERR-004, 404 ERR-008, 409 ERR-009
RF-05	`PUT /mascotas/{id}`	`ServicioMascotas.actualizar`	200 ficha actualizada	400 ERR-004, 403 ERR-003, 404 ERR-011
RF-05	`DELETE /mascotas/{id}?refugio_id={id}`	`ServicioMascotas.dar_de_baja`	204 sin cuerpo	400 ERR-004, 403 ERR-003, 404 ERR-011
RF-34	`GET /mascotas`	`ServicioCatalogo.listar`	200 página del catálogo	400 ERR-004

Formato único de error:

```
{ "detail": "mensaje legible", "codigo": "ERR-004" }
```

Código	HTTP	Significado
ERR-003	403	La ficha no pertenece al refugio indicado.
ERR-004	400	Datos de entrada inválidos (tipo, dominio, rango, formato).
ERR-006	409	Violación de integridad de la base.
ERR-008	404	El refugio no existe.
ERR-009	409	El refugio no está verificado.
ERR-011	404	La ficha no existe.

Las validaciones de forma también responden 400 con ERR-004 (no 422), para mantener un único contrato de error. Un fallo inesperado responde 500 con ERR-500 y se registra sin exponer detalles técnicos.

4.4. Ejemplos de los contratos

Crear una ficha (RF-05). Cuerpo de `POST /mascotas`:

```
{
  "refugio_id": 1,
  "nombre": "Nala",
  "raza": "Mestiza",
  "especie": "perro",
  "edad_meses": 24,
  "tamano": "mediano",
  "nivel_energia": "media",
  "experiencia_requerida": "basica",
  "apta_con_menores": true,
  "apta_con_mascotas": true,
  "historial_salud": "Vacunas al día",
  "historial_comportamiento": "Sociable",
  "fecha_publicacion": "2026-11-10",
  "fotografias": ["nala_01.jpg", "nala_02.jpg"]
}
```

Respuesta 201 Created (resumen; la respuesta real incluye todos los campos del contrato y el id depende de la base; en la base de prueba es 7):

```
{
  "id": 7,
  "refugio_id": 1,
  "nombre": "Nala",
  "especie": "perro",
  "edad_meses": 24,
  "tamano": "mediano",
  "estado": "disponible",
  "fecha_publicacion": "2026-11-10",
  "refugio_nombre": "Refugio Patitas",
  "refugio_localidad": "ENGATIVA",
  "fotografias": [
    { "id": 7, "archivo": "nala_01.jpg", "orden": 1 },
    { "id": 8, "archivo": "nala_02.jpg", "orden": 2 }
  ]
}
```

Consultar el catálogo (RF-34). `GET /mascotas?especie=perro&tamano=mediano` después de crear a Nala en la base de demostración:

```
{
  "total": 2,
  "pagina": 1,
  "por_pagina": 20,
  "mascotas": [
    {
      "id": 3, "nombre": "Rocky", "especie": "perro",
      "edad_meses": 60, "tamano": "mediano",
      "nivel_energia": "alta", "fecha_publicacion": "2026-10-03",
      "refugio_id": 1, "refugio_nombre": "Refugio Patitas",
      "refugio_localidad": "ENGATIVA", "foto_principal": "rocky_01.jpg"
    },
    {
      "id": 7, "nombre": "Nala", "especie": "perro",
      "edad_meses": 24, "tamano": "mediano",
      "nivel_energia": "media", "fecha_publicacion": "2026-11-10",
      "refugio_id": 1, "refugio_nombre": "Refugio Patitas",
      "refugio_localidad": "ENGATIVA", "foto_principal": "nala_01.jpg"
    }
  ]
}
```

Rechazo por refugio no verificado (RF-05):

```
{
  "detail": "el refugio 2 no está verificado y no puede publicar fichas",
  "codigo": "ERR-009"
}
```

Filtro válido sin coincidencias (RF-34): responde 200 con `total: 0` y `mascotas: []`. **Filtro inválido:** responde 400 con ERR-004. El detalle completo de cada contrato, con todos los campos y condiciones límite, está en `docs/contratos_api.md`.

4.5. Decisiones de diseño

- **Actor temporal.** La autenticación y los roles se integran en la semana 3. Mientras tanto, el refugio que actúa se envía como `refugio_id` en el cuerpo (POST/PUT) o en la consulta (DELETE).
- **Validación en dos niveles.** Pydantic valida la forma y el repositorio valida el dominio; ambos errores se traducen a ERR-004.
- **Base configurable.** La ruta de SQLite se lee de `DB_PATH` en cada petición, lo que permite aislar las pruebas sin tocar la base de desarrollo.
- **CORS preparado.** Los orígenes por defecto son `http://localhost:5173`, `http://127.0.0.1:5173` y `http://localhost:3000`, configurables con `CORS_ORIGINS`.
- **Baja lógica.** DELETE no borra la fila: cambia el estado a baja para conservar la trazabilidad y cumplir RN-15.
- **Sin datos personales.** El catálogo y la ficha pública solo exponen datos de la mascota y del refugio (nombre y localidad), no del adoptante.

4.6. Pruebas funcionales

```
python -m pytest -v
```

Resultado: **39/39 pruebas pasan** en aproximadamente 1,6 s (`evidencias/salida_pruebas_semana14.txt`). Aparece un aviso de deprecación de Starlette sobre `httpx`; no afecta los resultados.

RF	Escenario	Acción	Resultado obtenido	Evidencia
RF-05	Válido	POST con refugio verificado y 2 fotos	201, id 7, disponible, 2 fotos; verificado en SQLite desde otra conexión	`test_rf05_api.py`
RF-05	Rechazo	POST con refugio no verificado	409 ERR-009; 0 filas nuevas	`test_rf05_api.py`
RF-05	Entrada	POST con especie inválida	400 ERR-004; 0 filas nuevas	`test_rf05_api.py`
RF-05	Edición	PUT de ficha incompleta con una foto	200; pasa a disponible y aparece en el catálogo	`test_rf05_api.py`
RF-05	Baja	DELETE de una ficha propia	204; estado baja y ya no aparece en el catálogo	`test_rf05_api.py`
RF-34	Válido	GET del catálogo inicial	200, total 3, orden [1, 3, 2]	`test_rf34_api.py`
RF-34	Filtros	Especie, tamaño y rango de edad	[1, 3], [2] y [2]	`test_rf34_api.py`
RF-34	Paginación	por_pagina = 2	Página 1 [1, 3]; página 2 [2]; total 3	`test_rf34_api.py`
RF-34	Ausencia	Filtro válido sin coincidencias	200, total 0, lista vacía	`test_rf34_api.py`
RF-34	Entrada	especie = conejo	400 ERR-004	`test_rf34_api.py`

Distribución de las 39 pruebas:

Archivo	Propósito	Casos
`pruebas/test_rf05_api.py`	Funcionales de RF-05	8
`pruebas/test_rf34_api.py`	Funcionales de RF-34 y RNF-04	8
`pruebas/test_rnf10_integridad.py`	Matriz de entradas inválidas (RNF-10)	14
`pruebas/test_servicios_aislados.py`	Servicios con doble del repositorio (RNF-08)	9

4.7. Demostración HTTP real

```
python -m pruebas.demo_api
```

El script levanta Uvicorn en un hilo, usa un cliente `httpx` real y genera `evidencias/demo_api.txt` con las peticiones y respuestas completas: crear (RF-05), consultar catálogo (RF-34), actualizar (RF-05), dar de baja (RF-05), verificar que la ficha dada de baja sale del catálogo, catálogo vacío y filtro inválido. El resultado final es **CUMPLE**.

5. Verificación de requisitos no funcionales

RF / RNF	Calidad ISO/IEC 25010:2023	Prueba y meta	Resultado	Evidencia
RF-05, RF-34 / **RNF-05**	Eficiencia de desempeño / comportamiento temporal	1.000 fichas; 20 consultas consecutivas por HTTP; cada una ≤ 1 s	**20/20**; mínimo 2,62 ms; promedio 2,78 ms; p95 3,01 ms; máximo 3,05 ms	`evidencias/rnf_desempeno.txt`
RF-05, RF-34 / **RNF-08**	Mantenibilidad / capacidad para ser probado	Doble del repositorio, sin base real; cobertura ≥ 70 % en servicios	**9/9** pruebas aisladas; **100 %** de cobertura (33 sentencias)	`test_servicios_aislados.py`, `cobertura.txt`
RF-05, RF-34 / **RNF-10**	Seguridad / integridad	Matriz de entradas inválidas; 0 filas persistidas	**14/14** casos rechazados; 0 filas nuevas o modificadas	`test_rnf10_integridad.py`, `salida_pruebas_semana14.txt`
RF-05, RF-34 / **RNF-04**	Seguridad / confidencialidad	Revisar las claves del catálogo y de la ficha pública	**0** campos personales	`test_rf34_api.py`, `demo_api.txt`
RF-34 / **RNF-07**	Flexibilidad / escalabilidad	2.000 fichas y 5.000 adoptantes; p95 ≤ 1 s	*Pendiente* para la semana 3	—

5.1. Lectura honesta de RNF-05

La meta de la plantilla incluye 500 fichas y 50 usuarios concurrentes. En esta entrega se usó un volumen mayor (1.000 fichas), pero la medición es **secuencial**, no concurrente. Por eso el resultado se declara **cumplido en lo secuencial** y la prueba de 50 usuarios queda para la semana 3. No se rebajó el umbral: cada consulta se exigió ≤ 1 s.

5.2. Comandos de verificación

```
# RNF-08: cobertura de los servicios
python -m coverage run -m pytest -q
python -m coverage report -m

# RNF-05: 1.000 fichas y 20 consultas por HTTP real
python -m pruebas.rnf_desempeno
```

6. Reproducción de la entrega

6.1. Requisitos

- Python 3.12.10 (compatible con 3.10 o superior).
- SQLite 3.49.1 incluido en Python; sin servidor ni credenciales.
- Dependencias de `requirements.txt`: FastAPI, Uvicorn, pytest, httpx y coverage.

6.2. Instalación y ejecución

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

# Base de la API con el estado inicial conocido
python -m persistencia.inicializar_db

# Servidor local
uvicorn api.main:app --reload --port 8000
```

- Base URL: `http://127.0.0.1:8000/api/v1`
- Documentación interactiva: `http://127.0.0.1:8000/docs`
- OpenAPI: `http://127.0.0.1:8000/openapi.json`

Variable	Valor por defecto	Uso
`DB_PATH`	`evidencias/api.db`	Ruta de la base SQLite.
`CORS_ORIGINS`	`localhost:5173`, `127.0.0.1:5173` y `localhost:3000`	Orígenes permitidos para la interfaz.

6.3. Estructura relevante

```
proyecto_arq_sis_mascotas/
|-- api/                  # rutas, contratos y dependencias
|-- servicios/            # RF-05 y RF-34
|-- persistencia/         # esquema, seed, repositorio e inicializador
|-- pruebas/              # 39 pruebas y scripts de evidencia
|-- docs/
|   |-- contratos_api.md          # contratos completos
|   `-- informe_semanas_13_14.md  # informe en Markdown
|-- evidencias/           # pruebas, cobertura y desempeno
|-- plantilla/            # documentos formales
`-- README.md
```

6.4. Evidencias

Evidencia	Contenido
`evidencias/salida_pruebas.txt`	12/12 pruebas de persistencia (semana 13).
`evidencias/salida_pruebas_semana14.txt`	39/39 pruebas de la semana 14.
`evidencias/cobertura.txt`	Cobertura de servicios (RNF-08).
`evidencias/rnf_desempeno.txt`	20 consultas sobre 1.000 fichas (RNF-05).
`evidencias/demo_api.txt`	Peticiones y respuestas HTTP reales de ambos RF.
`evidencias/prueba.db`	Base reproducible de la semana 13.

7. Checklist de entrega

Criterio	Cumple	Evidencia
Los dos RF mantienen la prioridad y el alcance de la semana 13	**Sí**	Sección 2 y `plantilla_13_1.pdf`.
Ambos RF tienen servicios y endpoints identificables	**Sí**	`servicios/` y `api/main.py`.
Los contratos describen éxito, entrada inválida y errores	**Sí**	`docs/contratos_api.md` y `/docs`.
Hay resultado válido y rechazo/ausencia por cada RF	**Sí**	Sección 4.6.
Se verifica la persistencia real y la ausencia de escrituras indebidas	**Sí**	Pruebas con SQLite y conteos antes/después.
El SQL queda en persistencia y las reglas en servicio/dominio	**Sí**	Sección 4.1.
Cada RF se vincula con sus RNF y atributos ISO	**Sí**	Sección 5.
Los RNF previstos tienen medidas y evidencias	**Sí**	RNF-04, 05, 08 y 10; RNF-07 pendiente.
Se distinguen los incumplimientos y pendientes de la semana 3	**Sí**	Sección 8.
El README permite ejecutar y repetir las pruebas	**Sí**	Sección 6 y `README.md`.

8. Pendientes y relevo a la semana 3

Implementado y verificado

- Persistencia completa de RF-05 y RF-34 con integridad referencial.
- Servicios de aplicación y API REST con cuatro endpoints.
- Contratos documentados y OpenAPI generado por FastAPI.
- 39 pruebas funcionales, de integridad y de servicios aislados.
- RNF-04, RNF-05 (secuencial), RNF-08 y RNF-10 con evidencia reproducible.
- Demostración HTTP real de ambos RF.

Pendiente para la semana 3

- Autenticación, roles y reemplazo de `refugio_id` explícito por el usuario autenticado.
- Pantalla de ficha y catálogo (React/Vite) contra esta API.
- Prueba de concurrencia de RNF-05 (50 usuarios) y RNF-07 (2.000 fichas, 5.000 adoptantes).
- Migración y verificación en PostgreSQL (ADR-03/ADR-10).
- Almacenamiento binario de fotos (hoy se guarda la ruta).
- Rechazo automático de solicitudes pendientes al dar de baja (RN-15), fuera de este corte.

9. Trazabilidad

Elemento	Dónde se evidencia
RF-05	`servicios/servicio_mascotas.py`, `api/main.py` (POST/PUT/DELETE) y `pruebas/test_rf05_api.py`.
RF-34	`servicios/servicio_catalogo.py`, `api/main.py` (GET) y `pruebas/test_rf34_api.py`.
Contratos	`docs/contratos_api.md`, `/docs` y `/openapi.json`.
RNF-04	`test_catalogo_sin_datos_personales_rnf04`.
RNF-05	`pruebas/rnf_desempeno.py` y `evidencias/rnf_desempeno.txt`.
RNF-08	`pruebas/test_servicios_aislados.py` y `evidencias/cobertura.txt`.
RNF-10	`pruebas/test_rnf10_integridad.py` y `evidencias/salida_pruebas_semana14.txt`.
RNF-07	Pendiente de la semana 3.
Persistencia	`persistencia/schema.sql`, `persistencia/repositorio.py` y `evidencias/salida_pruebas.txt`.
Informe en Markdown	`docs/informe_semanas_13_14.md`.
Documento semana 13	`plantilla/plantilla_13_1.tex` y `plantilla/plantilla_13_1.pdf`.

Cierre

La semana 13 dejó una base reproducible y verificada; la semana 14 convirtió esa persistencia en servicios y una API con contratos estables. Las dos semanas mantienen el mismo alcance: publicar fichas (RF-05) y consultarlas (RF-34). La calidad se concentró en integridad, comportamiento temporal, capacidad de prueba y confidencialidad. Lo que falta —autenticación, interfaz, concurrencia y PostgreSQL— está identificado y se retoma en la semana 3.